LightGBM

A Highly Efficient Gradient Boosting Decision Tree

학번	212STG15	이름	서혜정
학번	212STG33	이름	황예진

Intro

LightGBM 은 gradient boosting 의 split point selecting 과정에서 모든 instance 에 대한 information gain 을 계산함에 따라 발생하는 time consuming 을 줄임으로써 Gradient boosting decision tree(GBDT)의 학습 속도를 개선하고자 한 방법이다.

GBDT 는 테이블 데이터에서 좋은 성능을 보이는 트리 기반 알고리즘이다. 하지만 feature dimension 이 높고 데이터 크기가 클 때 아직도 효율성과 확장성면에서 불만족스러운 모습이 나타난다. 모든 가능성이 있는 split points 에서 information gain 을 추정하기 위해 모든 데이터의 (instance) 인스턴스를 살펴보기 때문에, 많은 시간이 걸리기 때문이다. 이를 해결하기 위해 LGBM 를 제안한다.

LGBM 는 트리기반 알고리즘 중에서도 가벼워서(light) 대용량의 데이터를 빠르게 처리할 수 있다는 장점이 있다. 크게 두 가지 특징이 존재하는데, 하나는 Gradient based One Side Sampling(GOSS) 이고 다른 하나는 Exclusive Feature Bundling(EFB)이다. 이에 대해서는 뒤에서 자세히 설명한다.

해당 논문에서는 GOSS 와 EFB 방법을 적용한 GBDT 를 LightGBM 이라고 부른다. 논문에서 이 알고리즘은 기존 GBDT 에 비해 20 배 이상 빠른 훈련속도를 보여주면서 거의 동일한 성능을 보여준다.

GBDT

GBDT 는 decision tree 를 순차적으로 학습시키는 모형이다. 순차적으로 Negative Gradients 즉, 잔차(residual)를 계산하고 이를 target 으로 하는 새로운 decision tree 를 만들어 가는 방식으로 학습이 된다. 결국 decision tree 를 만드는데 중요한 것은 최적 분할점을 찾는 것인데 이 부분이 가장 시간이 오래 걸리는 부분이다.

논문에서는 이러한 decision tree 알고리즘으로 Histogram-based Algorithm 을 소개한다. 해당 알고리즘은 메모리나 속도 측면에서 효율적이다. Histogram-based Algorithm 은 정렬된 feature 값에서 분할 지점을 찾는 대신, 연속적인 feature 값들을 이산적인 bins 에 집어넣고 이 bins 을 사용하여 훈련 동안 feature histogram 을 구축한다.

구체적인 순서는 아래와 같다.

1. n_unique 계산
2. n_unique 와 설정한 bins 개수 중에 큰 값을 bins 의 개수로 지정
3. bin 의 개수만큼 histogram 생성. bin_edges 저장
4. bins 의 수 = split values 의 unique 수

GOSS

GOSS 는 데이터셋의 샘플 수를 줄이는 알고리즘으로서, Gradient 값이 큰 표본들을 일정 비율 추출하고 그렇지 않은 표본들을 일정비율로 random 하게 학습하는 방법이다. instance 의 gradient 가 크다는 것은, 해당 instance 를 구성하는 값(element)들의 변화가 손실함수에 큰 변화를 가져올 수 있음을 의미한다. 큰 Gradient 값을 가진 표본을 더 학습하는 것이 정보의 획득양이 더욱 많으며 반대로 Gradient 가 작은 표본들은 큰 영향을 미치지 않을 것이다. LightGBM 에서는 이런 instance 를 모델이 충분히 학습하지 못한(과소 적합 된) instance 라고 보고 있고, 따라서 이러한 instance 를 포함하면서 데이터셋의 sample 수를 줄이려고 시도한다.

GOSS 가 어떻게 데이터셋의 샘플 수를 줄이는지 그 과정을 간단히 살펴본다. 논문에서 제시한 알고리즘은 다음과 같다.

Algorithm 2: Gradient-based One-Side Sampling

Input: I: training data, d: iterations
Input: a: sampling ratio of large gradient data
Input: b: sampling ratio of small gradient data
Input: $loss$: loss function, L: weak learner
models ← { }, fact ← $\frac{1-a}{b}$
topN ← a × len(I) , randN ← b × len(I)
for $i = 1$ ***to*** d **do**
  preds ← models.predict(I)
  g ← $loss$(I, preds), w ← {1,1,...}
  sorted ← GetSortedIndices(abs(g))
  topSet ← sorted[1:topN]
  randSet ← RandomPick(sorted[topN:len(I)], randN)
  usedSet ← topSet + randSet
  w[randSet] × = fact ▷ Assign weight $fact$ to the small gradient data.
  newModel ← L(I[usedSet], − g[usedSet], w[usedSet])
  models.append(newModel)

알고리즘 1 Goss algorithm

GOSS 방법은 기울기가 큰 표본을 모두 sampling 하고 기울기가 작은 표본에 대해서는 random sampling 을 진행한다.

먼저 모든 데이터를 gradient 의 절대값을 기준으로 하여 정렬한 뒤, 전체 데이터에서 gradient 가 큰 a x 100%를 모두 선택한다. 이후 전체 데이터 중에서 b x 100%의 instance 를 random sampling 한다. Gradient 가 큰 a x 100%의 데이터와 random 하게 뽑힌 b x 100% 만큼의 데이터가 모델링에 사용되는 것이다. 이러한 과정을 통해 original data 를 크게 바꾸지 않으면서, 학습이 필요한 instance 에 집중할 수 있도록 하였다.

이때 표본들의 효과를 보정하고자 기울기가 작은 표본에 (1−a)/b 의 가중치를 주어 작은 gradient 를 가진 sampling 된 데이터를 증폭시킨다. 이렇게 되면 a:b 비율이 $a:\left(b*\frac{(1-a)}{b}\right)=a:1-a$ 가 된다. 따라서 초기 데이터 분포의 큰 변경 없고 original data 를 크게 바꾸지 않으면서, 학습이 필요한 instance 에 집중할 수 있다.

goss 는 전체 데이터의 일부를 사용하기 때문에 전체를 사용했을 때 보다 training accuracy 가 감소할 수 있다. 즉, information gain 의 분산이 달라질 수 있다. 다음 페이지는 수식을 통해 goss 의 과정을 통해 얻은 분산과 GBDT 의 분산에 큰 차이가 없음을 증명하였다.

[Goss 수식 참고]

GOSS 의 과정을 수식으로 표현해본다. 먼저 $\{x_1, \dots, x_n\}$: training set with i.i.d instances, $\{g_1, \dots, g_n\}$: negative gradients with respect to the output of model 이라고 할 때, splitting feature j at point d 에서 GBDT 의 variance of information gain after splitting 은 다음과 같다.

$$V_{j|O}(d) = \frac{1}{n_O}\left(\frac{\left(\sum_{\{x_i \in O: x_{ij} \le d\}} g_i\right)^2}{n_{l|O}^j(d)} + \frac{\left(\sum_{\{x_i \in O: x_{ij} > d\}} g_i\right)^2}{n_{r|O}^j(d)}\right)$$

이 때 O: decision tree 의 특정 node 에 속하는 training set, $n_o = \sum I[x_i \in O], n_{l|O}^j(d) = I[x_i \in O: x_{ij} \le d], n_{r|O}^j(d) = I[x_i \in O: x_{ij} > d]$을 뜻한다. $V_j(d)$를 가장 크게 만드는 feature j의 point d 가 최적의 split point 로 선택된다.

GOSS method 의 variance of information gain 은 다음과 같다.

$$\tilde{V}_j(d) = \frac{1}{n}\left(\frac{\left(\sum_{x_i \in A_l} g_i + \frac{1-a}{b}\sum_{x_i \in B_l} g_i\right)^2}{n_l^j(d)} + \frac{\left(\sum_{x_i \in A_r} g_i + \frac{1-a}{b}\sum_{x_i \in B_r} g_i\right)^2}{n_r^j(d)}\right)$$

이 때 A 는 gradient 가 큰 top-a x 100% instance 의 subset, B 는 A 를 제외하고 남은 A^c중에서 random 하게 뽑힌 b x $|A^c|$ 크기의 subset, $A_l = \{x_i \in A : x_{ij} \le d\}, A_r = \{x_i \in A : x_{ij} > d\}, B_l = \{x_i \in B : x_{ij} \le d\}, B_r = \{x_i \in B : x_{ij} > d\}$ 을 뜻하며, 식의 분자에 곱해지는 $\frac{1-a}{b}$는 sum of the gradients 를 normalize 하기 위한 coefficient 이다.

따라서 GOSS 는 특정 split point 에 대한 정확한 information gain 인 $V_j(d)$ 대신 전체 데이터의 smaller subset 으로 계산한 $\tilde{V}_j(d)$ 을 사용함으로써 계산 비용을 감소시킨다. 이 때 $V_j(d)$ 와 $\tilde{V}_j(d)$의 차이로 training accuracy 가 감소할 수 있는데, 본 논문에서는 두 값의 차이를 다음과 같이 정리한다.

GOSS 의 approximation error $\varepsilon(d) = |\tilde{V}_j(d) - V_j(d)|$, $g_l^{-j}(d) = \frac{\sum_{x_i \in (A \cup A^c)_l} |g_i|}{n_l^j(d)}, g_r^{-i}(d) = \frac{\sum_{x_i \in r} |g_i|}{n_r^j(d)}$일 때 $1-\delta$의 확률로 $\varepsilon(d)$의 upper bound 는 다음 과 같다.

$$\varepsilon(d) \le C_{a,b}^2 \frac{ln1}{\delta} \cdot \max\left\{\frac{1}{n_l^j(d)}, \frac{1}{n_r^j(d)}\right\} + 2DC_{a,b}\sqrt{\frac{\frac{ln1}{\delta}}{n}},$$

$$where\ c_{a,b} = \frac{1-a}{\sqrt{b}} \max_{x_i \in A^c} |g_i|, D = \max\left(g_l^{-j}(d), g_r^{-j}(d)\right)$$

식에 다르면 data size n 이 커질수록 ($\sqrt{\frac{\frac{ln1}{\delta}}{n}}$), 그리고 분할되는 두개의 leaf 에 데이터가 균등하게 분리될수록($\max\left\{\frac{1}{n_l^j(d)}, \frac{1}{n_r^j(d)}\right\}$) approximation error 가 작아진다.

Exclusive Feature Bundling (EFB)

EFB 는 데이터셋의 feature 개수를 줄이는 알고리즘이다. LightGBM 에서는 고차원의 데이터는 대체로 매우 sparse 하다고 보았다.

고차원 데이터일 수록 sparse 한 경우가 많다. 이때 sparse 란, 대부분의 값이 0 이거나 모두가 0 인 상황을 의미한다. 특히 sparse feature space 에선 변수의 대부분이 상호 배타적이다. 즉, 0 이 아닌 값을 동시에 갖지 않는다. 이러한 배타적 변수를 단일 변수로 묶을 수 있다. 구체적인 알고리즘은 아래와 같다.

Algorithm 3: Greedy Bundling

```
Input: F: features, K: max conflict count
Construct graph G
searchOrder ← G.sortByDegree()
bundles ← {}, bundlesConflict ← {}
for i in searchOrder do
    needNew ← True
    for j = 1 to len(bundles) do
        cnt ← ConflictCnt(bundles[j],F[i])
        if cnt + bundlesConflict[i] ≤ K then
            bundles[j].add(F[i]), needNew ← False
            break
    if needNew then
        Add F[i] as a new bundle to bundles
Output: bundles
```

Algorithm 4: Merge Exclusive Features

```
Input: numData: number of data
Input: F: One bundle of exclusive features
binRanges ← {0}, totalBin ← 0
for f in F do
    totalBin += f.numBin
    binRanges.append(totalBin)
newBin ← new  Bin(numData)
for i = 1 to numData do
    newBin[i] ← 0
    for j = 1 to len(F) do
        if F[j].bin[i] ≠ 0 then
            newBin[i] ← F[j].bin[i] + binRanges[j]
Output: newBin, binRanges
```

알고리즘 2

이때 해결해야 할 두 가지 문제가 있다. 첫째, 어떤 변수를 함께 묶을지 결정하는 것이다. 둘째, 어떻게 묶을까 방법에 관한 것이다.

논문에 따르면 첫번째 문제를 해결하기 위한 mutually exclusive 한 변수들을 bundle 로 묶는 문제는 NP-hard 이므로 greedy 알고리즘을 활용해야 한다. 논문에서는 최소의 bundle 로 묶는 문제를 NP-hard 의 대표적인 문제인 그래프 색칠 문제(Minimum vertex coloring)로 환원하여 이 문제가 NP-hard 임을 보였다.

*** Minimum vertex coloring**

Minimum vertex coloring 문제는 서로 연결되어 있는 두 점에 같은 색을 할당할 수 없을 때 최소로 필요한 색깔의 수를 구하는 문제이다. dataset 의 각 변수를 하나의 꼭지점으로 두고, mutually exclusive 하지 않은 변수들을 edge 로 연결한다. 따라서 서로 연결되어 있지 않은 mutually exclusive 한 변수들은 같은 색으로 색칠할 수 있는데, 이 문제에서 최소한으로 필요한 색깔의 수는 dataset 에서 묶을 수 있는 최소 bundle 의 개수와 대응된다.

따라서 최소한의 bundle 을 찾는 문제는 NP-hard 이므로 다항시간 내에 문제를 푸는 알고리즘이 존재하지 않으며, 이러한 이유로 논문에서는 greedy 알고리즘을 통해 문제에 대한 approximate solution 을 찾고자 하였다.

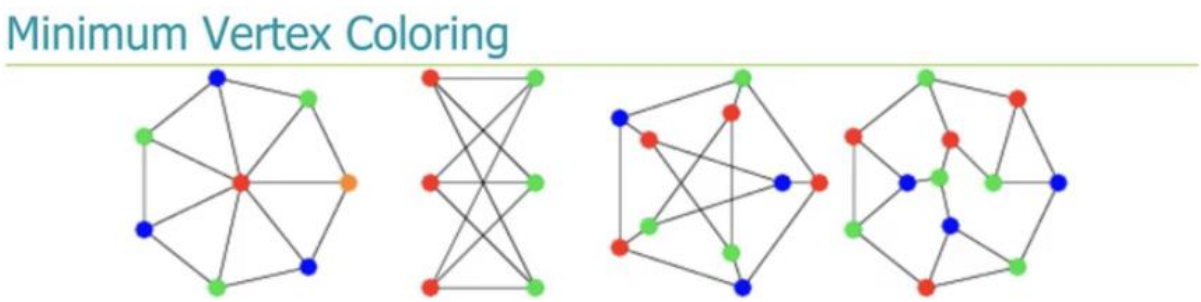

그림 1 Minimum vertex coloring 예시

첫번째 그림의 경우 8 개의 feature 가 4 개의 색이 다른 노드로 묶였다. 두번째 그림은 6 개의 feature가 2개의 bundle로 묶인다. 세번째 그림은 10개의 feature가 3개의 bundle로 묶인다. 마지막 그림의 경우, 12 개의 feature 가 3 개의 bundle 로 묶인다.

Greedy bundling 예시

Greedy bundling 의 구체적인 예는 아래와 같다. 이때 cut-off 는 hyperparameter 로 0.2 로 가정한다. 주어진 예시에서 instance는 10개이므로 10*0.2=2회 이상 겹치는 경우 bundling은 진행되지 않는다

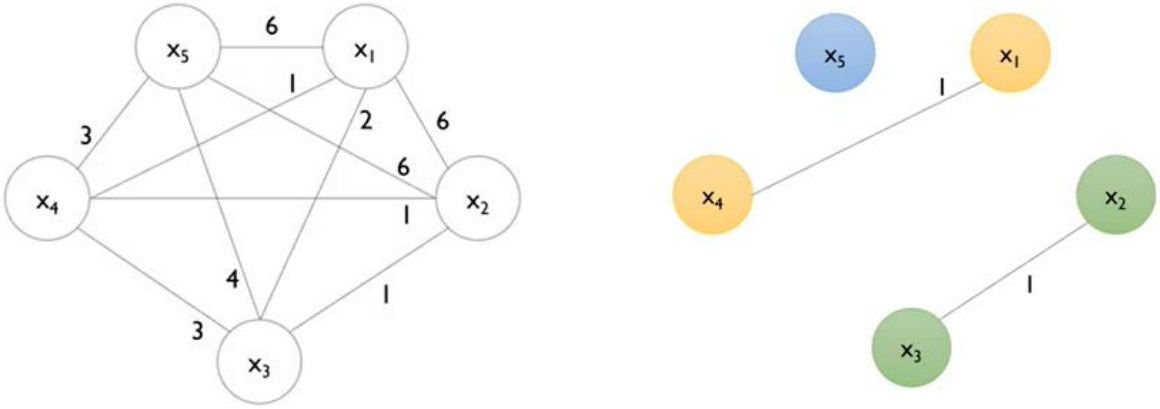

그림 2 greedy bundling 예시

- 각 변마다 가중치가 있는 그래프를 구성하는데, 여기서 가중치는 변수 간의 충돌(conflicts)을 의미한다. 여기서 충돌이란 non-zero value 가 동시에 존재하여 상호 배타적이지 않은 상황을 의미한다.
- 그래프 내에 있는 꼭짓점 차수에 따라 내림차순으로 변수들을 정렬한다.
- 정렬한 리스트에 있는 각 변수를 확인하면서 이들을 기존 묶음에 할당하거나, 새로운 묶음을 만든다. 이때 γ는 각 bundle 에서 최대 충돌 비율(cut-off)이다.

	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

	x_1	x_2	x_3	x_4	x_5
x_1	-	6	2	1	6
x_2	6	-	1	1	6
x_3	2		-	3	4
x_4	1	1	3	-	3
x_5	6	6	4	3	-

degree for choosing a starting point

	x_5	x_4	x_3	x_2	x_1
d	19	15	14	10	8

표 1

EFB 에서 해결해야하는 두번째 문제인 '어떻게 묶는가'는 offsets 를 feature 의 original value 에 더해주는 방식으로 해결된다. 변수 묶음으로부터 원래(original) 변수들의 값을 식별할 수 있어야 한다. Histogram-based 알고리즘은 변수의 연속적인 값 대신 이산적인 구간(bin)을 저장하므로, 배타적 변수들을 각각 다른 구간에 두어 변수 묶음을 구성할 수 있다. 위의 예시의 경우, 30 개중 3 개만 conflict 가 발생하여 원래 정보에서 손실이 발생한다.

	x_1	x_4	x_3	x_2	x_5
I_1	1	0	0	1	1
I_2	0	1	1	0	1
I_3	1	0	0	2	2
I_4	0	3	2	0	1
I_5	2	0	0	1	3
I_6	3	0	0	3	1
I_7	0	1	3	0	2
I_8	1	4	3	2	3
I_9	1	0	1	0	0
I_{10}	2	0	0	3	2

	x_5	x_{14}	x_{23}
I_1	1	1	1
I_2	1	4	4
I_3	2	1	2
I_4	1	6	5
I_5	3	2	1
I_6	1	3	3
I_7	2	4	6
I_8	3	1	2
I_9	0	1	4
I_{10}	2	2	3

표 2

LGBM 와 기존 tree 기반 알고리즘과의 차이점

기존의 tree 기반 알고리즘은 level wise(균형 트리 분할) 방식을 사용한다. 즉, 최대한 균형이 잡힌 트리를 유지하면서 분할하기 때문에 tree 의 깊이가 최소화가 될 수 있다. 하지만 이는 균형을 맞추기 위한 시간이 필요하다는 것을 의미한다.

LightGBM 의 leaf wise tree 분할 방식은 tree 의 균형을 고려하지 않고 최대 손실 값(max delta loss)를 가지는 leaf node 를 지속적으로 분할하면서 tree 의 깊이가 깊어지고 비 대칭적인 tree 가 생성된다. 이와 같이 최대 손실 값을 가지는 leaf node 를 반복 분할하는 방식은 loss 가 큰 node 를 선택해 loss 를 줄여나간다. 따라서 level wise tree 분할 방식보다 예측 오류 손실을 최소화할 수 있다.

LightGBM 은 대용량 데이터에서 학습 속도가 빠르고 categorical feature 들을 자동 변환과 최적분할을 해준다는 장점이 있다. 반면, 크기가 작은 dataset 을 사용할 경우 과적합의 가능성이 크다.

leaf wise vs level wise

만약 full tree 를 만들게 되면, best first (leaf-wise)와 depth-first (level wise)는 동일한 tree 가 만들어진다. 두 방법의 차이는 tree 를 확장해 나가는 순서(order)에 있다.

leaf wise 에서는 best node 가 먼저 확장된다. 이때 best node 란, 해당 노드의 분할(split)이 분할 가능한 모든 노드 중에서 불순도를 최대로 줄일 수 있는 노드를 의미한다.

이로 인해 항상 그런 것은 아니지만 종종 leaf wise 가 level wise 보다 lower error tree 를 더 빠르게 학습한다. node 의 수가 적을 때, leaf wise 는 level wise 보다 더 우수한 성능을 낼 것이다. 하지만 노드를 추가함에 따라 (stopping 이나 pruning 이 없다는 가정하에), 결과적으로 같은 tree 를 만들 것이기 문에 level wise 와 leaf wise 는 동일한 성능으로 수렴할 것이다.

참고) Shi, H. (2007). Best-first Decision Tree Learning (Thesis, Master of Science (MSc)). The University of Waikato, Hamilton, New Zealand. Retrieved from https://hdl.handle.net/10289/2317

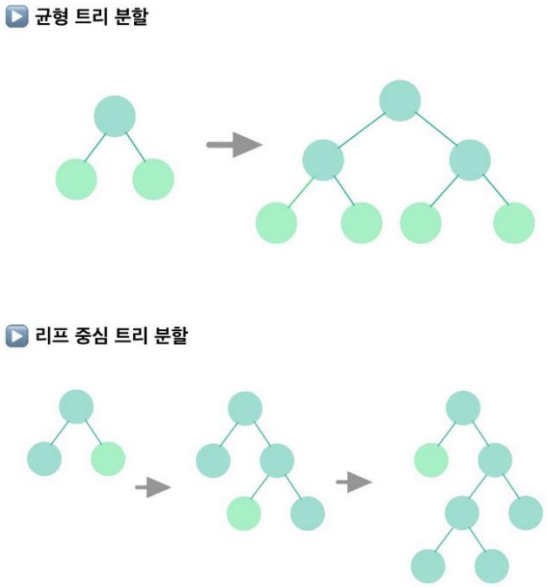

그림 3 균형 트리 분할과 리프 중심 트리 분할 비교

파라미터 튜닝

LightGBM은 leaf-wise tree growth algorithm을 사용하고, 다른 많은 일반적인 툴은 depth-wise tree growth을 사용한다. depth-wise에 비해 leaf-wise 알고리즘은 훨씬 빠르게 수렴할 수 있다. 하지만, 적절한 파라미터를 함께 사용하지 않으면 leaf-wise growth는 과적합을 일으킬 수 있다.

leaf wise tree를 사용하여 좋은 결과를 얻기 위해서는 아래의 파라미터들이 중요하다.

Tune Parameters for the Leaf-wise (Best-first) Tree

1. num_leaves

   이것은 트리 모델의 복잡성을 제어하기 위한 주요 매개변수이다. 이론적으로 num_defaults = 2^(max_depth)을 설정하여 depth-wise 트리와 동일한 leaf 개수를 얻을 수 있다. 하지만 이러한 간단한 변환은 실제로 좋은 선택이 아니다. leaf-wise tree의 경우 일반적으로 leaf 수를 고정한 depth-wise보다 깊이가 깊기 때문이다. 깊이가 제한되지 않으면 과적합이 발생할 수 있다.

   - num_leaves를 조정할 때는 2^(max_depth)보다 작게 해야 한다.

2. min_data_in_leaf

   이것은 leaf-wise tree에서 over-fitting을 막는 중요한 변수이다. 최적 값은 num_leaves와 training samples수에 따라 달라진다. 이 값을 큰 값으로 설정하면 트리가 너무 깊게 자라지 않도록 할 수 있지만 맞지 않을 수 있다. 실제로 대규모 데이터셋에는 수백 또는 수천 개로 설정해도 된다.

3. max_depth: tree의 깊이를 제한하는 변수이다.

모델 적합 시 조절가능한 파라미터는 다음과 같다.

Hyperparameter

parameter	default[range]	설명
objective	regression	문제 유형(regression, binary, multiclass 등)
learning_rate	0.1	트리 정보에 대한 학습률
num_iterations	100	boosting 반복 횟수
max_depth	-1(no limit)	최대 트리 깊이 제한
boosting	gbdt	부스팅 방법 rf / gbdt(traditional Gradient Boosting Decision Tree) / dart(dropout regression tree) / goss(gradient-based one-side sampling)
metric		evaluation set 평가 기준: binary classification이면 cross_entropy, log loss 등 regression이면 mse, rmse 등
early_stopping_round	0	validation set에 대한 평가가 몇번 동안 발전이 없을 때 그만둘지 제한
bagging_fraction	1	row sampling 비율. Boosting=GOSS 옵션을 쓸 경우 자동적으로 샘플링되므로 bagging_fraction이 1보다 작을 경우 에러가 발생함.
feature_fraction alias: sub_feature	1[0,1]	column sampling 비율(random forest의 column sampling과 같음)
scale_pos_weight	1	분류 문제에서 positive class에 weight를 부과. Imbalanced data에 시도해볼 수 있음.
min_data_in_leaf	20	leaf 하나당 최소 데이터 개수
min_gain_to_split	0	Splitting을 위해 필요한 최소한의 gain
num_leaves	31[1,131072]	한 트리 내 최대 leaf 수
max_bin	255	max number of bins that feature values will be bucketed in
top_rate	0.2[0,1]	used only in goss / the retain ratio of large gradient data
other_rate	0.1[0,1]	used only in goss / the retain ratio of small gradient data